

实验名称 实验1 不同类型复杂网络的构建及读取

一、实验内容及结果

- 1、构建 5 种不同类型的复杂网络并将构建的每个复杂网络保存到本地
- 2、从本地读入一个复杂网络，查阅 networkX-reference，输出网络的基本信息

源代码：

五种复杂网络：

```
# 无向有权图
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
G=nx.Graph()
G.add_nodes_from(["A","B","C","D","E","F"])
G.add_weighted_edges_from([("A","B",1),("A","D",2.5),("D","B",1.5),("C","B",1),("E","B",3),("E","C",1.5),("D","F",3),("D","E",1),("E","F",1)])
w=[G[e[0]][e[1]]['weight'] for e in G.edges()]
nx.draw(G,node_color="green",node_size=700,edge_color="black",width=w,with_labels=True)
plt.show()

source=[]
target=[]
for e in G.edges():
    source.append(e[0])
    target.append(e[1])

w=[G[e[0]][e[1]]['weight'] for e in G.edges()]
df1=pd.DataFrame({'source':source,'target':target,'weight':w})
df1.to_csv('d:/Netdata/save/net1.csv')
```

```
# 有向有权图
import networkx as nx
import matplotlib.pyplot as plt
DG=nx.DiGraph()
DG.add_nodes_from([1,2,3,4,5,6])
DG.add_weighted_edges_from([(1,2,1),(1,3,0.7),(2,4,3),(3,5,2),(4,6,0.6),(5,2,1.5),(5,4,0.8),(5,3,1.2),(3,6,1.2)])
```

```

elarge= [(u,v) for (u,v,d) in DG.edges(data=True) if d["weight"] > 1]
esmall= [(u,v) for (u,v,d) in DG.edges(data=True) if d["weight"] < 1]

pos=nx.spring_layout(DG,seed=700)

nx.draw_networkx_nodes(DG, pos, node_size=700,node_color="green")
nx.draw_networkx_labels(DG, pos, font_size=12, font_family="sans-serif")

nx.draw_networkx_edges(DG,pos,edgelist=elarge,width=3)
nx.draw_networkx_edges(DG,pos,edgelist=esmall,width=3,alpha=0.5,edge_color="b",style="dashed")

edge_labels=nx.get_edge_attributes(DG,"weight")
nx.draw_networkx_edge_labels(DG,pos,edge_labels)
ax=plt.gca()
ax.margins(0.08)
plt.axis("off")
plt.tight_layout()
plt.show()

source=[]
target=[]
for e in DG.edges():
    source.append(e[0])
    target.append(e[1])

w=[DG[e[0]][e[1]]["weight"] for e in DG.edges()]
df1=pd.DataFrame({'source':source,'target':target,'weight':w})
df1.to_csv('d:/Netdata/save/net2.csv')

```

E-R 随机图

```

import networkx as nx
import matplotlib.pyplot as plt
random_graph = nx.erdos_renyi_graph(10, 0.4)
nx.draw(random_graph, with_labels=True,node_size=700,node_color="green",edge_color="black", width=3)
plt.show()
df1=pd.DataFrame({'source':source,'target':target})
df1.to_csv('d:/Netdata/save/net3.csv')

```

K 近邻耦合图

```

import networkx as nx

```

```

import matplotlib.pyplot as plt
n=10
G=nx.watts_strogatz_graph(n,6,0.0) #k=6
plt.figure(figsize=(4,4))
nx.draw(G,pos=nx.circular_layout(G),node_size=700,node_color="green",with
_labels=True,width=3)

source=[]
target=[]
for e in G.edges():
    source.append(e[0])
    target.append(e[1])

df1=pd.DataFrame({'source':source,'target':target})
df1.to_csv('d:/Netdata/save/net4.csv')

```

```

#karate club
import networkx as nx
import matplotlib.pyplot as plt
G=nx.karate_club_graph()
nx.draw(G,with_labels=True,node_color="green")
plt.show()
"""nodes = list(G.nodes())
print("节点数量:", len(nodes))
print("节点列表:", nodes)
edges = list(G.edges())
print("边数量:", len(edges))
print("边列表:", edges)
degree = dict(G.degree())
print("节点度:", degree)"""
source=[]
target=[]
for e in G.edges():
    source.append(e[0])
    target.append(e[1])

df1=pd.DataFrame({'source':source,'target':target})
df1.to_csv('d:\\Netdata\\save\\karate.csv')

```

读入复杂网络：

```
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_csv("D:\\Netdata\\save\\karate.csv")
G=nx.from_pandas_edgelist(df, 'source', 'target', create_using =nx.Graph())

nx.draw(G,with_labels=True,node_color="green")
plt.show()
```

#结构特征

```
print(f"节点数: {G.number_of_nodes()}")
print(f"边数: {G.number_of_edges()}")
print(f"密度: {nx.density(G)}") #实际边数/可能存在边数
print(f"平均度: {sum(dict(G.degree()).values()) / G.number_of_nodes()}")
print(f"直径: {nx.diameter(G)}") #最大最短距离的长度
print(f"平均聚类系数: {nx.average_clustering(G)}")
print(f"平均最短路径长度: {nx.average_shortest_path_length(G)}")
```

#度分布

```
degrees = [G.degree(n) for n in G.nodes()]
print("节点度:", degrees)
degree_counts = {}
for degree in degrees:
    if degree in degree_counts:
        degree_counts[degree] += 1
    else:
        degree_counts[degree] = 1
```

绘制度分布直方图

```
plt.bar(degree_counts.keys(), degree_counts.values())
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Degree Distribution')
plt.show()
```

连通性

```
is_connected = nx.is_connected(G)
print("是否连通:", is_connected)
```

中心性最高的两个节点

```
degree Centrality = nx.degree Centrality(G)
d_sorted_nodes = sorted(degree Centrality.items(), key=lambda x: x[1], reverse=True)
d_top_two_nodes =d_sorted_nodes[:2]
```

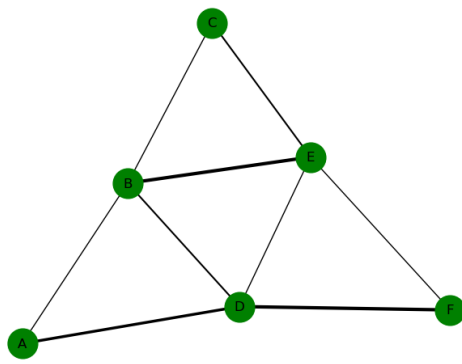
```

for i, (node, centrality) in enumerate(d_top_two_nodes):
    print("节点 {} 的度中心性为 {}".format(node, centrality))
print("-----")
betweenness Centrality = nx.betweenness Centrality(G)
b_sorted_nodes = sorted(betweenness Centrality.items(), key=lambda x: x[1], reverse=True)
b_top_two_nodes = b_sorted_nodes[:2]
for i, (node, centrality) in enumerate(b_top_two_nodes):
    print("节点 {} 的介数中心性为 {}".format(node, centrality))

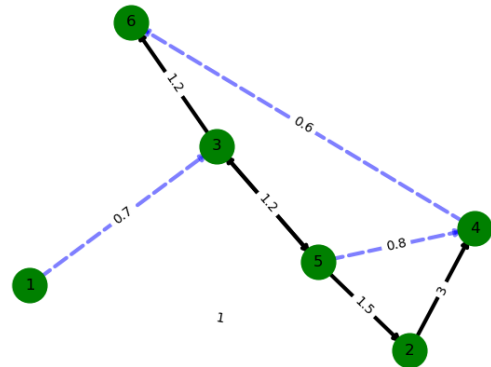
```

运行结果（截图）：

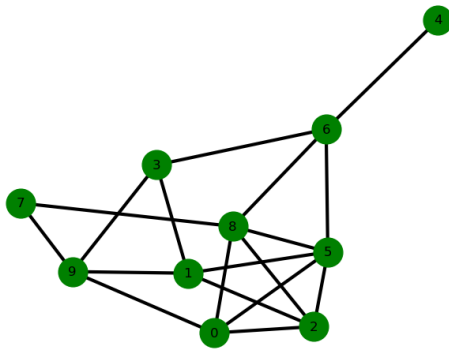
Task1:



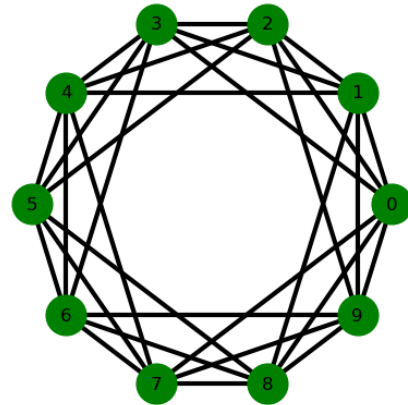
有权无向图



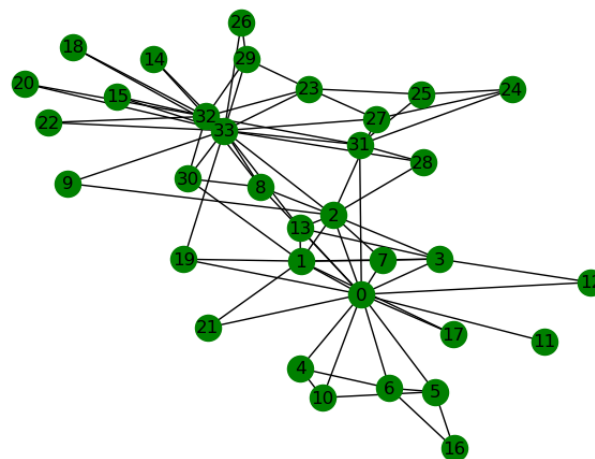
有权有向图



E-R 随机图

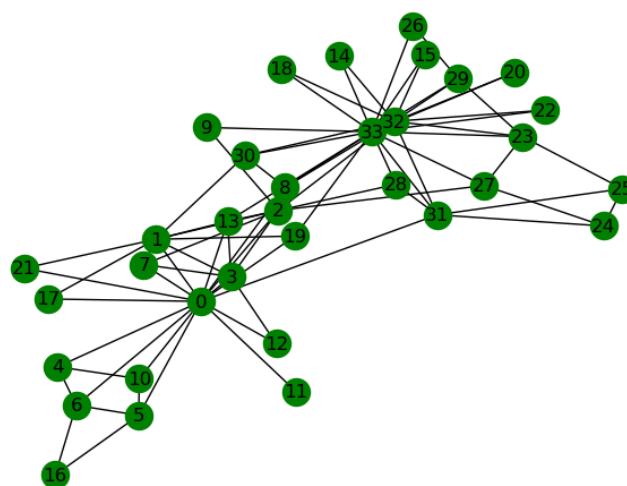


K 近邻耦合图 (k=6)

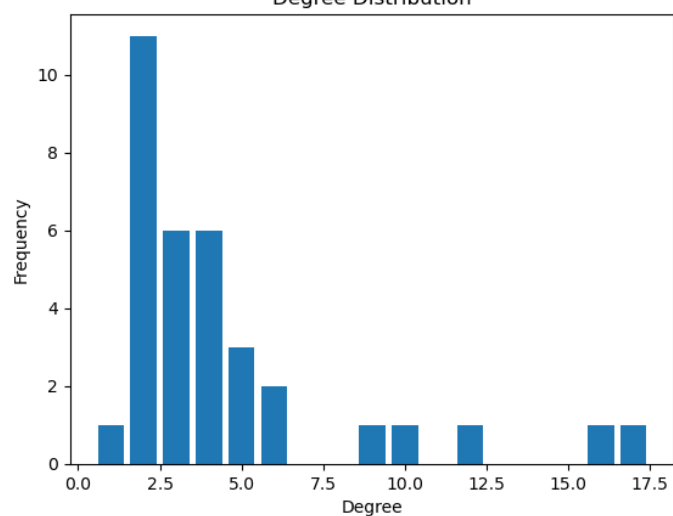


Karate-club

Task2:



csv 文件读入网络
Degree Distribution



网络节点度分布

节点数：34
边数：78
密度：0.13903743315508021
平均度：4.588235294117647
直径：5
平均聚类系数：0.5706384782076823
平均最短路径长度：2.408199643493761

网络结构特征

是否连通：True
节点 33 的度中心性为 0.5151515151515151
节点 0 的度中心性为 0.48484848484848486

节点 0 的介数中心性为 0.43763528138528146
节点 33 的介数中心性为 0.30407497594997596

网络连通性、度中心性与介数中心性最高的两点及其值